Project ID: 22-26/IT/G03

A PROJECT REPORT ON

Real-Time Collaborative Code Editor

Submitted
for the Partial fulfillment of award of
Bachelor of Technology
in
Information Technology
by

Aryan Tyagi (2200270130031)

Ananya Jaiswal (2200270130017)

Ashutosh Singh (2200270130035)

Harsh Kumar (2200270130062)

Under the guidance of

Mr. Nikhil Kumar

AJAY KUMAR GARG ENGINEERING COLLEGE,

GHAZIABAD

November 9, 2025

Declaration

We here by declare that the work presented in this report, entitled as **Real-Time Collaborative Code Editor**, was carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors and sources for all the words, ideas, diagrams, graphics, computer programs, experiments, and results that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors and sources.

We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, We shall be fully responsible and answerable.

Name : Aryan Tyagi
Roll No. : 2200270130031

Name : Ananya Jaiswal
Roll No. : 2200270130017

Name : Ashutosh Singh
Roll No. : 2200270130035

Name : Harsh Kumar
Roll No. : 2200270130062

Certificate

This is to certify that the report entitled **Real-Time Collaborative Code Editor** submitted by **Aryan Tyagi (2200270130031), Ananya Jaiswal (2200270130017), Ashutosh Singh (2200270130035), Harsh Kumar (2200270130062)** to the Dr. A. P. J. Abdul Kalam Technical University, Lucknow (U.P.) in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Information Technology is a bonafide record of the project work carried out by him/her under my/our guidance and supervision. This report in any form has not been submitted to any other university or institute for any purpose, to the best of my knowledge.

Mr. Nikhil Kumar
Assistant Professor
Department of Information Technology
Ajay Kumar Garg Engineering College

Dr Rahul Sharma
Professor & HOD
Department of Information Technology
Ajay Kumar Garg Engineering College

Place: Ghaziabad
November 9, 2025

Acknowledgements

We would like to express our thanks to all the people who have helped bring this project to the stage of fulfillment. We would wish to put on record very special thanks to my major project mentor, **Mr. Nikhil Kumar**, for the support, guidance, encouragement, and some very valuable insight that she guided us in the entire process. Her mentorship has been very pivotal in terms of shaping of our project and leading us toward excellence.
We would like to appreciate our Head of the Department, **Dr. Rahul Sharma**, who had provided us with the wherewithals and put us into an environment that would bring out such innovation towards learning. We would also want to appreciate our teachers and faculty members for all that they share at this crucial juncture in our academic career. We wish to appreciate many more for: helped out or, with their presence, indirectly made contributions to this project.

Contents

Declaration **i**

Certificate **ii**

Acknowledgements **iii**

List of Figures **vi**

1 Introduction **1**
1.1 Problem Statement of Project 1
1.2 Scope of Project 2
1.3 Detail of Problem Domain 3
1.4 Gantt Chart 4
1.5 System Requirements 5
1.6 Project Report Outline 6

2 Literature Review **7**
2.1 Related Study 7
2.2 Research Gaps 10
2.3 Objective of Project 11

3 Methodology Used **12**

4 Designing of Project **18**
4.1 0-Level DFD 18
4.2 1-Level DFD 19
4.3 Use Case Diagram 20

5 Detailed Designing of Project **23**
5.1 Class Diagram 23
5.2 Entity-Relationship Diagram (ERD) 25
5.3 Activity Diagram 27
5.4 Sequence Diagram 29

Bibliography **31**

6 Appendix A **33**

7 Appendix B **34**

8 Appendix C **35**

List of Figures

1.1 Gantt Chart 4

3.1 Flow Diagram of Proposed Realtime Collaborative Code Editor 13

4.1 0-level DFD 18
4.2 1-level DFD 19
4.3 Use Case Diagram 21

5.1 Class Diagram 24
5.2 ER Diagram 26
5.3 Activity Diagram 28
5.4 Sequence Diagram 29

Chapter 1

Introduction

1.1 Problem Statement of Project

Programming is, in the current digital age, regarded as a core skill in all fields but is still taught in an old-fashioned manner in schools and colleges. Although programming languages such as Python, Java, and C++ are part of the syllabus, the teaching method is mostly theoretical. Students are typically forced to code on paper or work on old labs rather than coding live. This gap results in a lack of real-world exposure, weak debugging skills, and poor readiness for industry expectations.

Another significant issue is the infrastructural problem encountered by most institutions. Computer labs are based on resource-intensive installations, constant updates, and high upkeep, hence not being viable for use in schools with limited infrastructure.

Lack of teamwork restricts learning potential. Existing tools and IDEs are created for single-user consumption, not considering the collaborative nature of contemporary software development.Instructors do not have equipment to track students' progress in real-time or offer guided, interactive coding assistance throughout classroom sessions.

Lastly, affordability and accessibility are still key issues. Most sophisticated coding environments are expensive to subscribe to or need expensive systems, making students from poor backgrounds unable to access them.There is therefore an urgent need for a light, browser-based, collaborative code editor that runs on low-end machines and encourages collaboration, facilitates teacher–student interaction, and includes AI support for real-time feedback. The platform would fill the gaps of accessibility, inclusivity, and pedagogy in today's programming education.

1.2 Scope of Project

1. **Accessible Coding Platform :** The project will provide a browser-based, light editor that does not need to be installed. It will operate effi- ciently even on low-cost machines, making coding accessible in institutions with limited resources. This allows all students to practice coding without technical or financial constraints.

2. **Real-Time Collaboration :** The platform will enable multiple users to code, edit, and debug real-time. Students can work on teamwork, peer learning and collaborative problem-solving. This mirrors contemporary software development methodology and readies students for professional careers.

3. **Teacher Integration :** Teachers can design code sessions and observe student work in real time. They can provide immediate feedback and assist students during practice in the classroom. This enhances classroom interaction and provides optimum instructional assistance.

4. **AI-Powered Assistance :** The editor will feature AI capabilities such as contextual suggestions, code proposal, and debugging assistance. It will serve a virtual tutor that allows students to study autonomously. With the help of teachers, it forms a dual guidance system for students.

5. **Inclusivity and Future Growth :** The project focuses on affordability, inclusivity, and good digital habits. Monitoring screen time and secure communication will be incorporated for safe use. In the future, the system can be extended to hackathons, coding challenges, and professional Colab .

1.3 Detail of Problem Domain

1. **Educational Technology (EdTech) -** The project falls under the domain of EdTech, where technology is applied to improve learning. Programming education is one of the key focus areas, but tools available today are old or restrictive. A contemporary, collaborative platform is required to enhance student participation and results.

2. **Computer Science Education -** Conventional programs in programming cover syntax and theory but not real-world exposure. Students are seldom exposed to teamwork-based programming or live problem solving. This industry-cum-curriculum gap is what constitutes a key challenge in the field.

3. **Collaborative Software Development -** In the real world, coding is seldom a solo endeavor—it's collaborative. Even most schools and colleges do not expose students to such experiences. An area challenge here is creating platforms that introduce collaborative methodologies in learning.

4. **AI-Assisted Learning -** Artificial Intelligence is revolutionizing education through customized learning assistance. Yet, its adoption in coding education is minimal at the school and under-graduate level. Placing AI inside a collaborative editor fills this gap in the problem domain.

5. **Accessibility and Inclusivity in Learning -** Most current coding platforms demand expensive subscriptions or top-tier devices. This makes them inaccessible for institutions and students from weaker socio-economic backgrounds. The domain challenge is to develop an affordable, lightweight solution that caters to all learners.

1.4 Gantt Chart

A Gantt chart is a visual project management tool that displays tasks, their duration, and dependencies on a timeline. It helps in tracking progress, scheduling tasks, and improving team coordination.

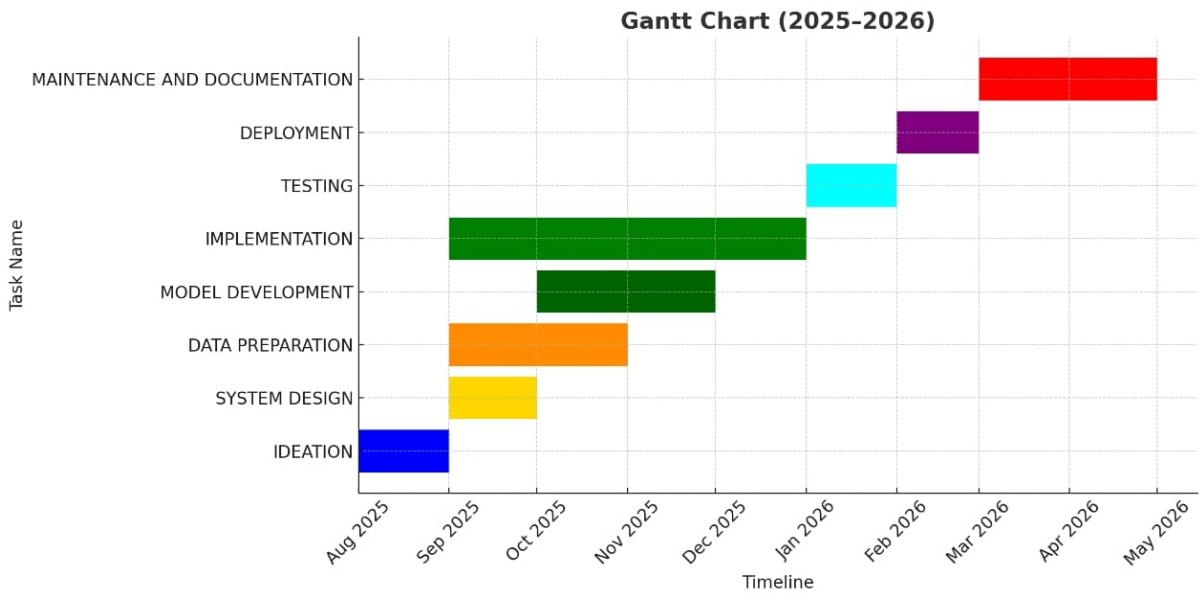

Figure 1.1: Gantt Chart

1.5 System Requirements

Hardware Requirements :

1. **Processor:** Minimum Intel i3 or equivalent, Recommended Intel i5/i7 for smooth multitasking.
2. **RAM:** Minimum 4 GB, Recommended 8 GB or above for handling real-time collaboration.
3. **Storage:** At least 250 GB HDD/SSD, with 1 GB free space for project-related files.
4. **Display:** Minimum resolution 1366 × 768, Recommended Full HD for better UI experience.
5. **Internet:** Stable broadband connection (minimum 2 Mbps, recommended 10 Mbps).

Software Requirements :

1. **Operating System:** Windows 10 or higher / Linux / macOS (cross-platform).
2. **Programming Languages:** JavaScript (for frontend & backend), Node.js runtime.
3. **Frameworks/Libraries:** React.js for frontend, Express.js for backend, Socket.IO or WebRTC for real-time communication.
4. **Database:** MongoDB or Firebase (for storing user data and projects).
5. **Editor/IDE Support:** Browser-based Monaco Editor (used in VS Code).
6. **AI Support:** Integration with OpenAI/LLM APIs for chatbot assistance.
7. **Others:** Web browser (Google Chrome/Firefox/Edge).

1.6 Project Report Outline

Chapter 2 – Literature Review

Discusses related studies, existing solutions, and research gaps. Highlights the need for the proposed system and justifies its development.

Chapter 3 – Methodology Used

Explains the approach, system architecture, diagrams (class, use case, ERD), and tools used. Describes how the project requirements are translated into a working solution.

Chapter 4 – Designing of Project

Focuses on detailed design aspects including UI/UX, flowcharts, and screen designs. Demonstrates how each module and component is structured and implemented.

Chapter 5 – Detailed Designing of Project

Provides an in-depth explanation of functionalities, coding logic, and workflows. Includes detailed explanations of real-time collaboration, AI integration, and other key features.

Chapter 6 – Results and Discussion

Presents screenshots, outputs, and discussions of the implemented system. Analyzes how the objectives have been achieved and evaluates performance.

Chapter 7 – Further Work

Discusses possible enhancements, future extensions, and improvements for the system.

Appendices

Includes supplementary materials such as code snippets, diagrams, and additional references.

Chapter 2

Literature Review

2.1 Related Study

Aditya Kurniawan et al. (2015) [1]: This paper introduces CodeR, a web-based real-time code editor application designed to facilitate collaborative programming. Utilizing WebSocket technology, CodeR enables users to write, execute, and display code results through a terminal, all within a shared workspace. The application supports real-time collaboration, chat features, and terminal building, catering to programming languages such as C, C++, and Java. The study emphasizes the importance of web-based tools in modern software development, allowing developers to collaborate seamlessly regardless of location.

Shivam Chowdhury et al. (2025) [2]:The research focuses on the design and development of a real-time code editor application aimed at enhancing collaboration during technical interviews. By employing WebSocket technology, the application allows users to collaborate in real-time on coding tasks. It features a workspace for creating, running, and building source code, along with real-time chat and terminal functionalities. The study highlights the potential of such platforms in remote technical assessments, providing an efficient and accessible environment for both interviewers and candidates.

Khushwant Virdi et al. (2023) [3]: This paper delves into the evolution and challenges of collaborative code editors, particularly in the context of real-time multi-user coding and knowledge sharing. The authors review the underlying technologies and protocols employed in these platforms, such as WebSocket communication and Operational Transformation (OT) algorithms. They identify persistent issues like latency, synchronization challenges, and conflict resolution complexities. The study also presents a

prototype collaborative code editor, evaluating its performance and usability compared to existing platforms, and offers recommendations for future research and development in this field.

Mohd Saad Ansari (2025) [4]: The study presents a minimalistic, browser-based real-time collaborative code editor built using the MERN stack and Socket.IO for real-time communication. The platform enables multiple users to write, view, and execute code together seamlessly, making it suitable for peer coding, interviews, and learning scenarios. The research emphasizes the importance of simplicity and performance in designing collaborative tools, aiming to provide an intuitive and efficient environment for real-time programming collaboration.

Anup Lal Yadav et al. (2024) [5]: This research explores the design and development of an advanced collaborative code editor prototype that integrates WebSocket communication and OT algorithms to facilitate real-time multi-user coding. The authors conduct a comprehensive literature review to understand the evolution of collaborative code editors and identify key technologies and protocols. They also address challenges such as latency and synchronization issues, proposing solutions to enhance collaboration and knowledge sharing among distributed development teams.

Krishna Aher Prof. Jagruti R. Mahajan (2023) [6]: This paper presents the development of a real-time collaborative code editor web application using React.js, Node.js, and Socket.io. The application enables developers to collaborate on code in real-time, regardless of their location. Key features include syntax highlighting, automatic code formatting, code versioning, and error highlighting. The study emphasizes the importance of such tools in enhancing productivity and collaboration among development teams. Future prospects include integrating artificial intelligence (AI) for intelligent code suggestions and auto-completion, machine learning (ML) algorithms for automatic error detection and fixing, and virtual reality (VR) or augmented reality (AR) for immersive coding experiences.

Anup Lal Yadav et al. (2023) [7]:This research paper delves into the intricate fabric of collaborative code editing, aiming to unravel the underlying technologies, challenges, and advancements in this field. The study begins with a development in this field, emphasizing the need for continued innovation and the exploration of novel strategies to address

the persistent challenges faced in collaborative code editing environments. The authors discuss key technologies such as WebSocket communication and Operational Transformation (OT) algorithms, and propose solutions to enhance collaboration and knowledge sharing among distributed development teams.

R. Pathak et al. (2024) [8]: The authors explore the trends, advantages, and future prospects of coding and collaborative applications, demonstrating the transformative potential of real-time tools in computer development. The study introduces V-Code, an advanced application for remote coding using technologies such as Nuxt.js and Python Flask. V-Code incorporates OpenAI machine learning models to provide intelligent code suggestions and auto-completion, enhancing the coding experience. The research highlights the growing importance of real-time collaborative code editors in modern software development.

2.2 Research Gaps

Despite several studies and developments in collaborative coding platforms and intelligent code editors, there remain critical gaps that justify the need for further research:

1. Limited Real-Time Collaboration for Students : Existing platforms primarily focus on professional developers or general-purpose collaborative editors. There is insufficient research on designing collaborative coding environments specifically tailored for school or college students, addressing their learning pace and programming comprehension.

2. Integration of AI Assistance : While AI-based code suggestion tools like GitHub Copilot exist, few studies integrate AI-assisted learning features directly within a real-time collaborative environment to guide students or novice coders during live coding sessions.

3. Cross-Platform Accessibility : Most solutions either focus on web-based or desktop-based editors. There is a lack of seamless cross-platform real-time editors that synchronize edits across devices without lag or data inconsistency.

4. Scalability and Performance Evaluation : Existing research often neglects detailed evaluation of scalability for multiple concurrent users. Handling real-time synchronization for a large number of students in a classroom setting remains an underexplored area.

5. User Engagement and Interaction Analysis : Few studies examine how collaborative coding platforms impact student engagement, peer interaction, and learning outcomes in real-time coding environments. Quantitative and qualitative assessments in academic settings are limited.

6. Security and Privacy in Educational Contexts : While collaboration tools exist, research on secure real-time coding for students, including data privacy, role-based access, and session management, is sparse.

7. Customization for Learning Objectives : Most collaborative editors do not provide adaptive interfaces or features that align with specific learning objectives, such as beginner-friendly hints, syntax highlighting for specific languages, or interactive debugging exercises.

2.3 Objective of Project

The primary objective of this study is to design and develop a Realtime Collaborative Code Editor that enhances coding skills and learning experiences for students. The study aims to address the limitations identified in the literature and research gaps. The objectives are as followed:

1. Develop a Real-Time Collaborative Environment : For creating a web-based platform that allows multiple users to write, edit, and execute code simultaneously with minimal latency and real-time synchronization.

2. Integrate AI-Assisted Learning Features : To incorporate intelligent code suggestions, error detection, and hints to support novice learners and enhance coding efficiency.

3. Ensure Cross-Platform Accessibility : designing the editor in a way that it functions seamlessly across devices, including desktops, laptops, and tablets, without compromising performance.

4. Evaluate User Engagement and Learning Outcomes : To assess how collaborative coding affects student interaction, engagement, and overall learning, using qualitative and quantitative metrics.

5. Maintain Security and Data Privacy : To implement secure user authentication, role-based access, and data protection mechanisms suitable for educational environments.

6. Provide Customizable Learning Tools : To allow users and educators to customize the interface, programming languages, and learning modules according to specific course objectives.

Chapter 3

Methodology Used

Public Datasets : One should begin to look for public data-sets that have something to do with brain MRI scans.

This section describes the methodology proposed to develop a Realtime Collaborative Code Editor. The study aims to provide students with an interactive environment where multiple users can code simultaneously, receive AI-assisted guidance, and collaborate in real time. The system is designed to improve coding learning, foster teamwork, and provide immediate feedback on code execution.

The designed system takes input from users through a web interface and performs multiple stages, including real-time synchronization, AI assistance, syntax handling, and code execution. Each process is described

in detail below.

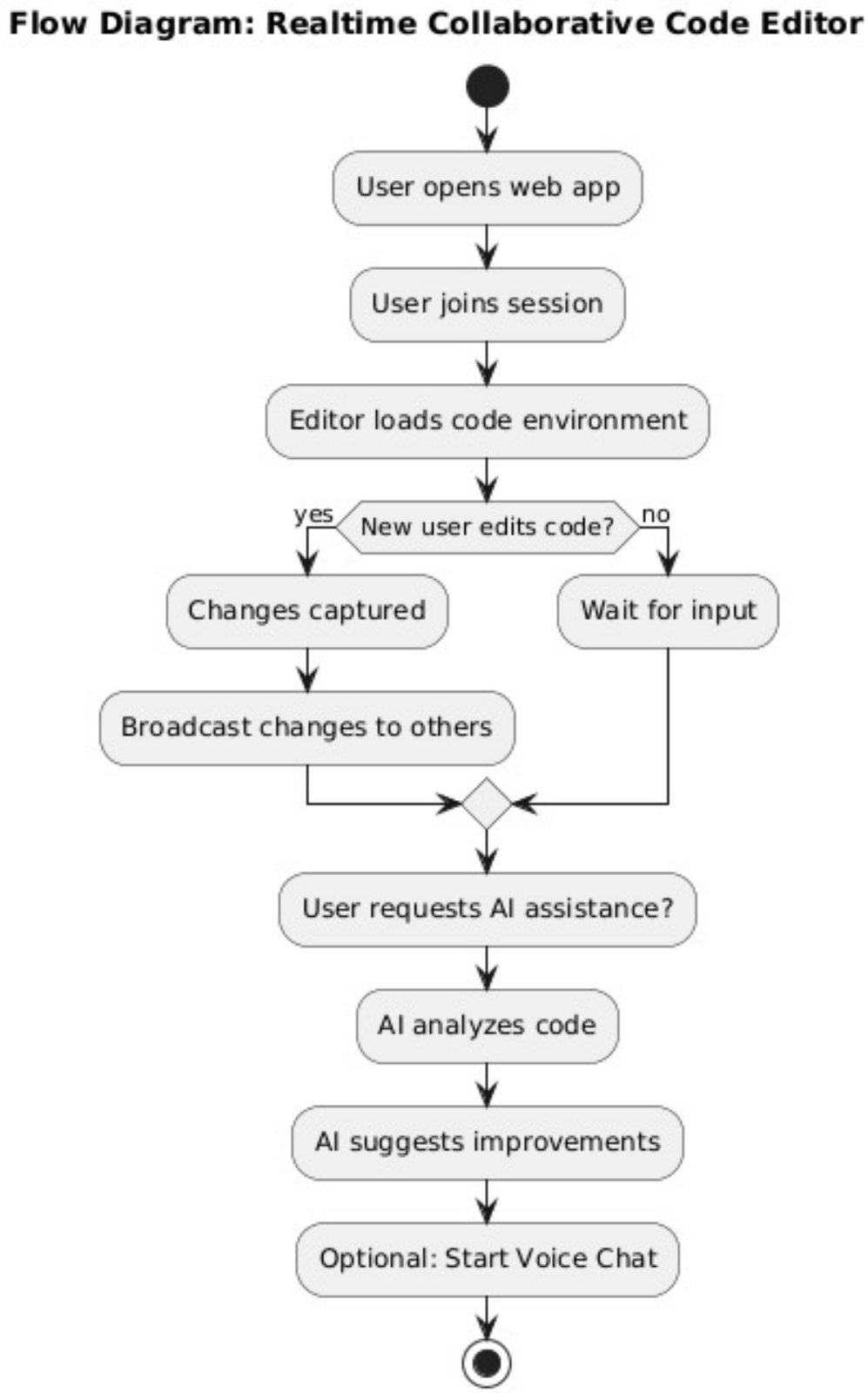

Figure 3.1: Flow Diagram of Proposed Realtime Collaborative Code Editor

3.1 Code Input and User Interaction

Code input is the initial step in the proposed system. The editor is accessed by users via a web-based interface and users start entering code using their favored programming language (Python, Java, C++, etc.). The editor records every keystroke and transmits it to the server for real-time synchronization.
This brings about the ability for many users to work on the same code simultaneously without overwriting other changes. Features like line numbers, autocomplete, and editor themes are supported to improve usability and accessibility.
Example: When three students are working on a Python script collaboratively, any alteration created by one of them is immediately updated in the editor windows of the other two students.

3.2 Real-Time Synchronization

Real-time synchronization is essential in providing consistency among multiple users. The system relies on Conflict-Free Replicated Data Types (CRDTs) realized through Yjs, integrated with WebSocket or Socket.IO for low-latency communications.
• CRDT prevents concurrent edits from conflicting.
• WebSocket offers a two-way communication pipe between the client and server.
This approach eliminates conventional locking mechanisms,resulting in seamless collaboration and minimizing delays even when several users are editing the same line of code. Illustration: If a student inserts a new function while another is editing a variable within the same script, CRDT automatically merges the changes without losing any of them.

- CRDT ensures that simultaneous edits do not conflict.
- WebSocket provides a two-way communication channel between the client and server.

  This method avoids traditional locking mechanisms, making collaboration seamless and reducing delays even when multiple users are editing the same line of code. Example: If one student adds a new function while another modifies a variable in the same script, CRDT merges the changes automatically without losing any edits.

3.3 Pre-processing and Syntax Handling

Before code can be executed or AI suggestions applied, it undergoes preprocessing:

- Syntax Highlighting: Differentiates keywords, variables, and functions using colors for easier reading.
- Error Detection: Identifies syntax errors in real time and highlights them to the user.
- Code Formatting: Ensures consistent indentation and spacing.

  Preprocessing not only improves readability but also enhances the accuracy of AI suggestions and prevents runtime errors.

  Example: Indentation errors in Python are flagged immediately, allowing students to correct them before execution.

3.4 AI-Assisted Code Suggestions

The AI module provides:

- Autocomplete for functions and variables
- Contextual code hints based on programming patterns
- Error detection and suggestions for correction

  The AI is designed to support novice learners by providing hints and examples without giving direct answers. This encourages learning through exploration while reducing frustration caused by frequent coding errors.

  Example: When writing a loop, the AI can suggest the correct syntax and possible conditions based on previously written code.

3.5 Feature Extraction and Analysis

Feature extraction involves identifying key components of the code:

- Functions and methods
- Loops and control structures
- Variable usage and scope

  This information is used by the AI for predictive suggestions, collaborative analytics, and conflict resolution. It also helps in generating reports on individual or team performance during coding sessions.

Example: Tracking how often a student uses loops incorrectly can help the AI provide targeted hints for improvement.

3.6 Comparison and Conflict Resolution

Changes made by different users are compared and merged using CRDT. Conflict resolution ensures:

- No edits are lost
- All participants see the same version of the code
- Collaboration is smooth and uninterrupted

Example: If two users edit the same line simultaneously, CRDT merges the changes in a logical sequence without overwriting any edits.

3.7 Code Execution and Output Display

The system executes the code in a secure sandbox environment to prevent security risks. Outputs are displayed to all users in real time. Features include:

- Console output panel for program results
- Highlighting runtime errors
- Supporting multiple programming languages

Example: When a Python script prints numbers from 1 to 10, all collaborators see the output instantly, and any error (like division by zero) is highlighted immediately.

3.8 Dataset

To evaluate the system, sample programs and exercises are used:

- Beginner-level programs (loops, conditional statements)
- Intermediate collaborative exercises
- Complex scripts for AI-assisted testing

These datasets help measure latency, synchronization accuracy, AI effectiveness, and user engagement.

3.9 Preprocessing of Code

Preprocessing ensures the system works efficiently:

- Validation: Detect syntax errors before execution
- Normalization: Standardize code formatting
- Error Handling: Identify potential runtime issues

3.10 Testing and Evaluation Sets

The system is tested using three sets:

- Training Set: Programs used to train the AI module
- Validation Set: Programs used to tune AI suggestions and real-time features
- Testing Set: Independent programs used to evaluate system performance and user experience

3.11 AI Module Implementation

The AI module uses pre-trained models or APIs such as OpenAI Codex to provide:

- Code completion
- Syntax suggestions
- Debugging hints

This feature encourages collaborative learning and reduces the cognitive load on beginners.

3.12 Summary of Methodology

The proposed methodology ensures that:

- Multiple users can collaborate in real time without conflicts
- AI-assisted suggestions improve learning outcomes
- Code execution is safe and immediate

Features like syntax highlighting, error detection, and preprocessing enhance usability

Chapter 4

Designing of Project

4.1 0-Level DFD

Figure 4.1 enlists the general flow of a Real-time Collaborative Code Editor with AI Assistance, considering the interaction between users, collaborators, and the system itself.

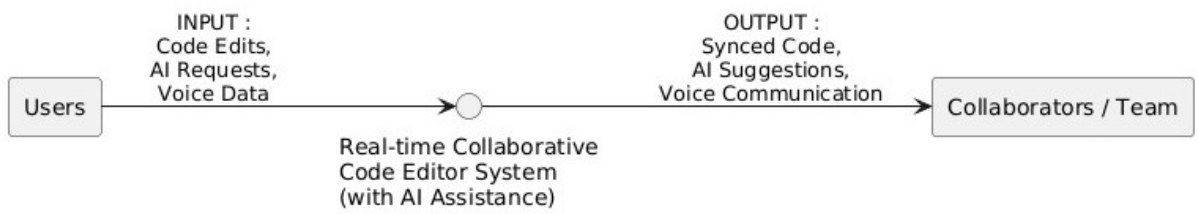

Figure 4.1: 0-level DFD

1. Users : The process begins with users who actively write, edit, and manage code. They provide inputs such as code edits, AI requests for suggestions or debugging help, and voice data for communication. These inputs form the raw data for the system to process in real time.

2. Real-time Collaborative Code Editor System (with AI Assistance) : This is the central system of the process. It incorporates user inputs, synchronizes code across multiple collaborators, and leverages AI techniques for features like intelligent code completion, error detection, and real-time suggestions. The system also integrates voice communication tools to facilitate seamless interaction between collaborators. Based on processing, it produces outputs such as synced code, AI-driven suggestions, voice-based communication, and version-controlled updates.

3. Collaborators : The processed outputs are delivered to other team members or collaborators. In this way, they can view synchronized code in real time, receive AI-driven insights, and communicate efficiently.

This ensures smooth collaboration, enhanced productivity, and accurate teamwork in coding projects.

4.2 1-Level DFD

Figure 4.2 is a 1-level DFD for a real-time collaborative code editor with AI assistance. This system manages user interactions, collaborative editing, and AI integration.

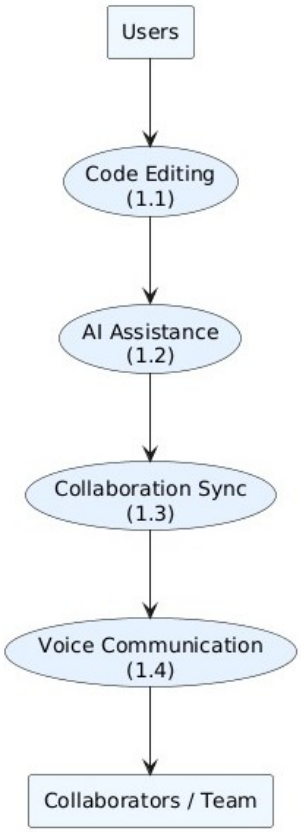

Figure 4.2: 1-level DFD

1. Code Editing (1.1) : This process is responsible for handling user code inputs within the collaborative editor. Users can write, modify, and update their code in real time. The edits are first captured by the system and stored in the shared workspace. These edits are then prepared to be sent for AI Assistance or synchronized with other collaborators. The output of this process is the updated Code Snippet that serves as the central element for collaboration.

2. AI Assistance (1.2) : This process provides AI-powered support to users during code editing. It takes the Current Code Snippet or a specific User Query as input and forwards it to the AI Model API. The AI

processes the request and generates appropriate outputs, such as Code Suggestions, Explanations, or Debugging Help. These results are then returned to the user within the editor, enhancing productivity and reducing errors.

3. Collaboration Sync (1.3): This process enables real-time synchronization of code edits among multiple users. When code changes are made by a user, they are transmitted to this process. Synchronization algorithms like Operational Transformation (OT) or CRDT merge the incoming edits into a single consistent Collaborative Codebase. The merged code is then distributed back to all participants, ensuring that every user sees the same real-time updates in the editor.

4. Voice Communication (1.4): This process manages the real-time voice and communication features of the system. It allows users to initiate and participate in voice conversations during a coding session. The system handles inputs such as Voice Data and Call Requests, which are routed to other team members. The outputs are Voice Streams and Notifications, enabling seamless spoken collaboration alongside coding.

4.3 Use Case Diagram

Below Figure 4.3 depicts a use case for a system of Realtime Collaborative Code Editor involving two main actors, namely the User and the AI Assistance Module.

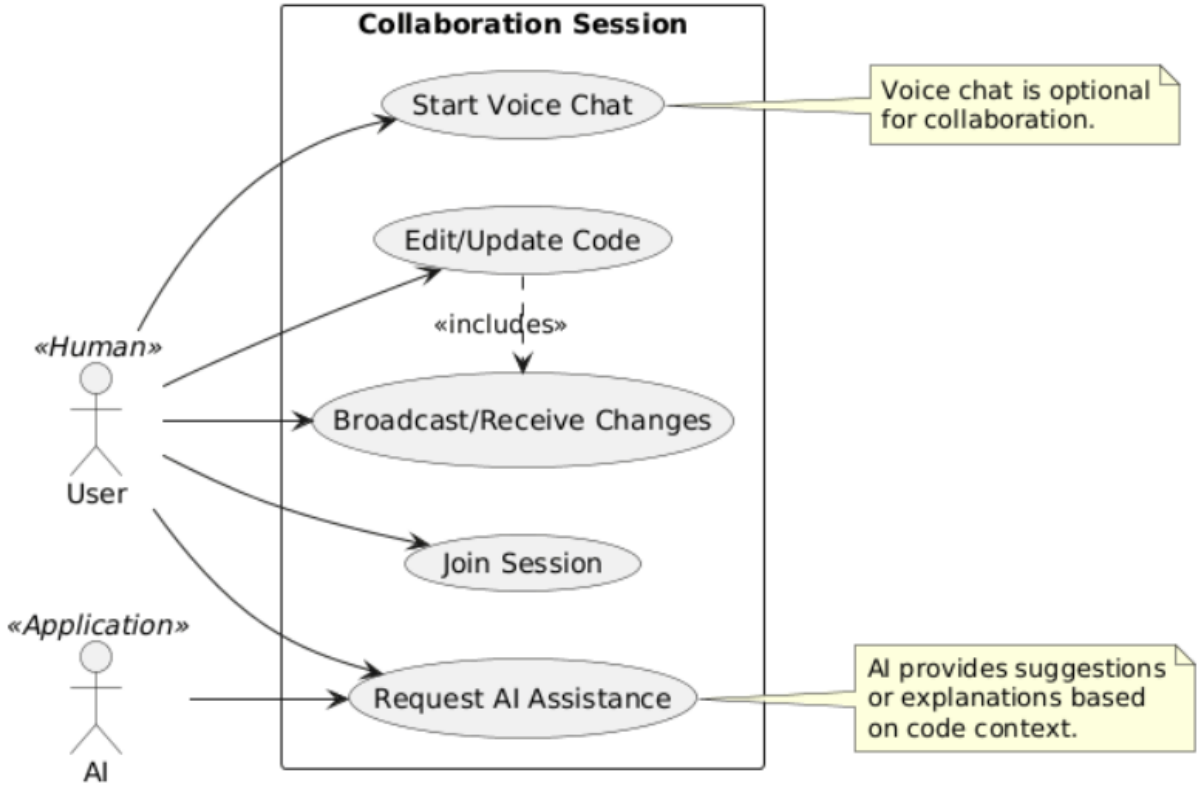

Figure 4.3: Use Case Diagram

1. **Role of User**

The user interacts directly with the system through the collaborative interface. Some of the most important activities include:

- **Join Session:** The user connects to an existing coding session or creates a new one to collaborate with peers.
- **Edit/Update Code:** The user writes, modifies, or deletes code within the shared editor.
- **Broadcast Changes:** The system automatically synchronizes updates so that all participants see the latest version of the code in real time.
- **Start Voice Chat:**The user can initiate a voice chat session with collaborators for better communication and teamwork.

2. **Role of AI Assistance Module**

The AI module acts as a supportive actor in the system to enhance learning and efficiency. Its roles include:

- **Request AI Assistance:** The AI provides real-time suggestions, error detection, or explanations based on the user's code.

- **Context Awareness:** AI analyzes the coding environment to ensure suggestions are relevant to the ongoing task.
- **Learning Support:** AI can offer explanations of programming concepts, helping beginners understand the logic behind the code.

Chapter 5

Detailed Designing of Project

5.1 Class Diagram

The following Figure 5.1represents a detailed class structure of a realtime collaborative code editor system. This system can be broken down into core functional modules, communication modules, and AI integration. The classes describe the methods, attributes, and their interrelationships that enable smooth collaborative coding, session management, and intelligent code assistance. Class of User allows individuals to interact with the system. The major methods include:

- **connect() :** Establishes connection to the collaboration session.
- **sendEdit():** Sends an edit operation made by the user.
- **receiveUpdate():** Receives updates from other participants in the session.

1. Collaboration Session : This class represents an active collaboration environment. It manages multiple users and handles real-time updates. Major functions include:

- **startSession() :** Initializes a new session with a unique session ID.
- **broadcastEdit() :** Broadcasts an edit operation to all participants.
- **resolveConflicts():** Handles synchronization and conflict resolution during collaborative editing.

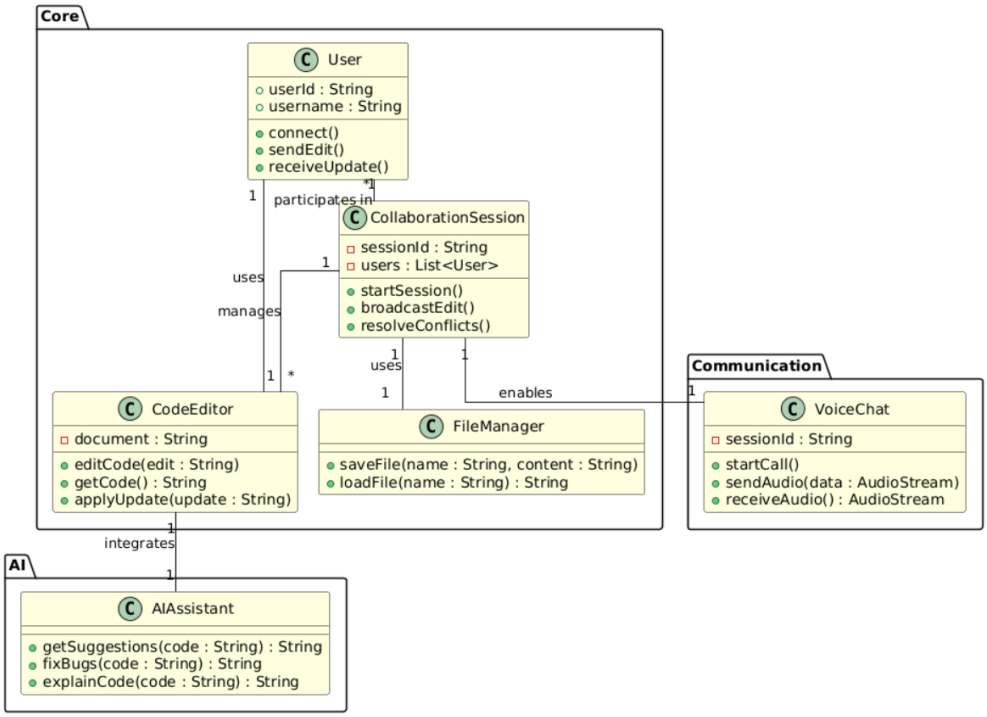

Figure 5.1: Class Diagram

2. Code Editor : This class is responsible for direct interaction with the code document. It supports editing, applying updates, and retrieving code. Its methods are:

- **editCode(edit: String) :** Applies user edits to the code.
- **getCode() :** Returns the latest version of the code.
- **applyUpdate(update: String) :** Integrates received edits from other users.

3. File Manager : This class manages persistence of code files, allowing saving and loading of code artifacts. Its methods include:

- **saveFile(name: String, content: String) :** Saves the current document.
- **loadFile(name: String) :** Loads a stored document for editing.

4. Voice Chat (Communication Module) : This optional class enables real-time verbal communication among participants. Its methods include:

- **startCall() :** Initiates a voice session tied to the collaboration session ID.

- **sendAudio(data: AudioStream):** Sends audio stream during a session.
- **receiveAudio() :** Receives audio stream from other participants.

5. AI Assistant (AI Module) : This class integrates intelligent features into the code editor. It provides support for bug fixing, suggestions, and explanations. Major methods include:

- **getSuggestions(code: String) :** Suggests code improvements.
- **fixBugs(code: String):** Identifies and fixes possible code errors.
- **explainCode(code: String) :** Provides explanation of selected code segments.

5.2 Entity-Relationship Diagram (ERD)

This figure 5.2attached image is the Entity-Relationship (ER) Diagram of a system that supports real-time collaborative code editing with AI assistance and voice communication.

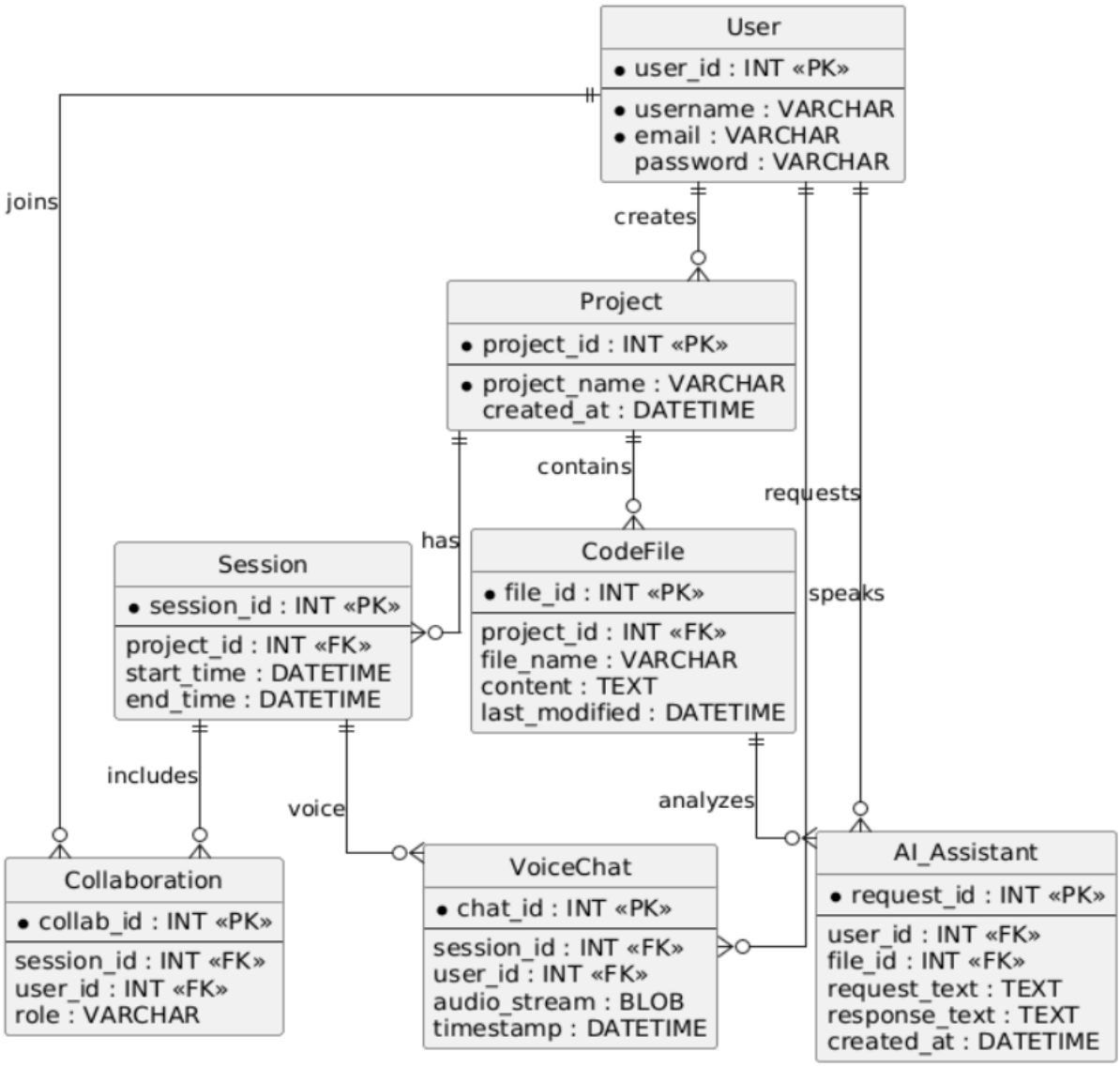

Figure 5.2: ER Diagram

It is actually the description of the relations which may exist between several entities in the process, like users, collaboration sessions, project files, AI requests, and communication activities.

1. User : Attributes: User ID, Name, Email, Role.

Each user represents an individual who can join sessions, edit code, send messages, request AI suggestions, and participate in calls.

2. Collaboration Session : Attributes: Session ID, Title, Created At, Status. The session acts as the central unit where multiple users collaborate. Within a session, files are shared, changes are tracked, AI requests are submitted, and communication takes place.

3. Project File : Attributes: File ID, Session ID (FK), Filename, Language, Last Modified. Files are contained within a session, and each file can have multiple versions and edits performed by different users.

4. Code Change : Attributes: Change ID, File ID (FK), User ID (FK), Timestamp, Diff. Code changes represent edits made to a project file by users. Each change is logged with details of the difference, time, and author.
5. AI Request : Attributes: Request ID, Session ID (FK), User ID (FK), Request Type, Created At. A user within a session can make an AI request, such as asking for suggestions, refactoring, or explanations of code.
6. AI Suggestion : Attributes: Suggestion ID, Request ID (FK), Suggestion Text, Confidence. AI suggestions are generated as responses to AI requests. They provide recommendations or improvements with a confidence score.
7. Voice Call : Attributes: Call ID, Session ID (FK), Started At, Ended At. Users within a session can initiate and participate in voice calls to enhance real-time communication.
8. Chat Message : Attributes: Message ID, Session ID (FK), User ID (FK), Content, Created At. Messages allow participants to exchange text-based communication within the session. They can include normal text, code snippets, or system-generated notifications.

5.3 Activity Diagram

In this figure5.3 the schematic explains the process by which a real-time collaborative code editor manages editing, AI assistance, and communication among users. It has different significant steps on its workflow:
1. Join Collaboration Session: The process begins when a user joins an existing collaboration session. This allows them to be part of the shared coding workspace.
2. Edit Code: The user performs editing or updates the source code inside the editor. Any modification made here is an active contribution to the project.

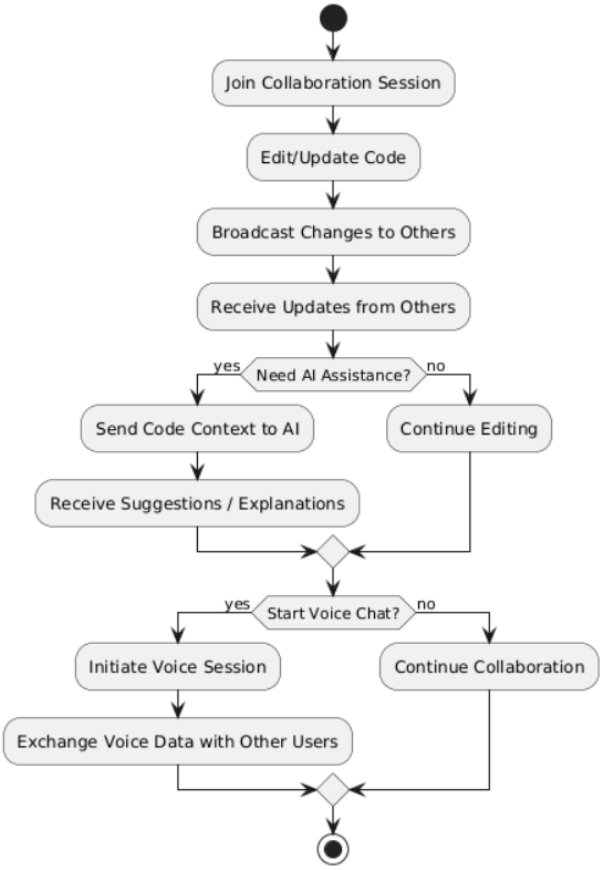

Figure 5.3: Activity Diagram

3. Broadcast Changes to Others: Once the code is modified, the changes are automatically broadcast to all connected participants in real-time to ensure synchronization.
4. Receive Updates from Others: The user also receives code updates performed by others. This step ensures that everyone in the session has the most up-to-date version of the project.
5. Need AI Assistance?: At this stage, the system checks whether the user requires AI support. If no, the user continues editing. If yes, the code context is sent to the AI module.
6. Send Code Context to AI: When AI assistance is requested, the system sends the relevant code context to the AI for analysis.
7. Receive Suggestions: The AI processes the input and returns suggestions, improvements, or explanations that can guide the user in resolving issues or optimizing their code.
8. Start Voice Chat?: After receiving AI help, or while continuing collaboration, the system provides an option to start a voice chat. If the user does not initiate voice chat, collaboration continues as usual.
9. Initiate Voice Session: If the user chooses to initiate a voice chat, a

voice session is established to facilitate faster communication among collaborators.
10.Exchange Voice Data with Other Users: During the active voice session, voice data is transmitted and received among users in real-time, ensuring smooth communication.
11. Continue Collaboration: At the end, whether through text editing, AI assistance, or voice chat, collaboration continues until the project goal is achieved.

5.4 Sequence Diagram

The following diagram 5.4 gives a sequence flow in a collaborative coding session, using techniques both in real-time synchronization and AI assistance.
There are phases, actually several within the process, from the original image up to the final result. For this answer, I elaborate step by step.

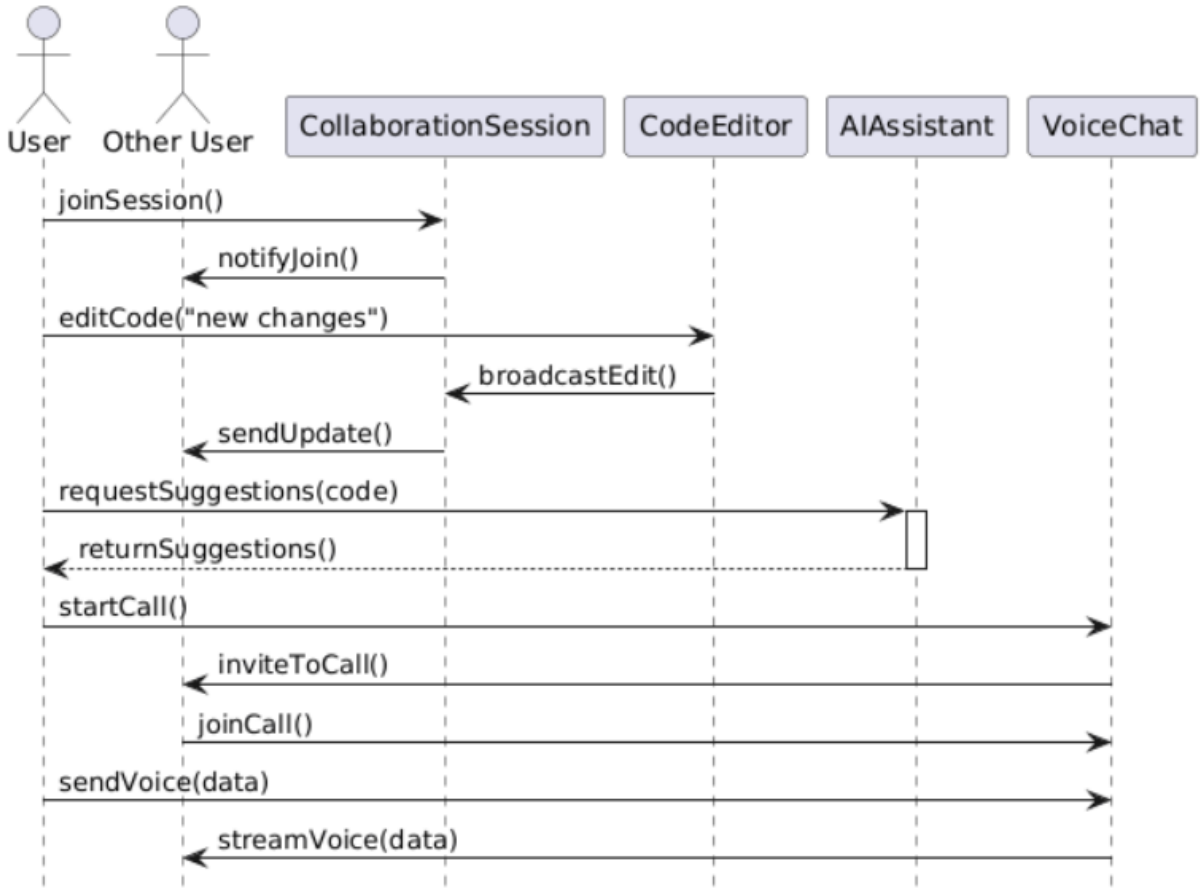

Figure 5.4: Sequence Diagram

1. Join Session: The process begins when a user joins a collaborative session. This triggers a notification to other participants, informing them that a new member has joined.
2. Code Editing and Updates: When the user edits the code (for example, adding "new changes"), the system broadcasts these edits through the Code Editor. The Collaboration Session ensures that updates are synchronized across all connected users.
3. AI Suggestions: The user can request suggestions by sending the current code context to the AI Assistant. The assistant processes the code and returns suggestions, which are then delivered back to the user for improved productivity.
4. Voice Call Initialization: To enhance collaboration, the user may initiate a voice call. The Collaboration Session sends an invitation to other users, who can then accept and join the call.
5. Voice Streaming: Once the call is established, users can exchange audio data. The voice data is sent by one user, streamed via the Voice Chat system, and received by all participants in real time.
6. Result: The entire flow integrates collaborative coding, AI-driven code suggestions, and real-time voice communication, ensuring an efficient and seamless teamwork experience.

Bibliography

[1] Grover, S., & Pea, R. Computational thinking in K–12: A review of the state of the field. *Educational Researcher*, 47(3), 166–186, 2018.

[2] Sentance, S., & Waite, J. Teaching computer programming in schools: Pedagogical approaches and challenges. *International Journal of Computer Science Education in Schools*, 1(1), 1–12, 2017.

[3] Wing, J. Computational thinking as an educational goal. *Communications of the ACM*, 60(10), 28–30, 2017.

[4] Luxton-Reilly, A., et al. The influence of collaboration in programming education. *Proceedings of the ACM Technical Symposium on Computer Science Education*, 1054–1060, 2018.

[5] Lye, S. Y., & Koh, J. H. L. Review of learning approaches in programming education. *Educational Technology*, 56(5), 44–52, 2016.

[6] Replit. Multiplayer coding platform overview. Available at: `https://replit.com/site/multiplayer`, 2021.

[7] Microsoft. Visual Studio Live Share Documentation. Available at: `https://learn.microsoft.com/en-us/visualstudio/liveshare/`, 2020.

[8] Kasneci, E., et al. ChatGPT and education: Opportunities and challenges. *British Journal of Educational Technology*, 2023.

[9] Vaithilingam, P., et al. Expectations and experiences of AI code assistants. *CHI Conference on Human Factors in Computing Systems*, 2022.

[10] Livingstone, S., & Third, A. Children and digital media: Impacts and ethical design. *UNICEF Office of Research*, 2017.

[11] Kafai, Y., & Burke, Q. Connected Code: Why Children Need to Learn Programming. MIT Press, 2016.

[12] Resnick, M., et al. Scratch: Programming for all. *Communications of the ACM*, 60(11), 60–67, 2017.

[13] Fagerlund, J., et al. Artificial Intelligence in programming education: A systematic review. *Computers and Education: Artificial Intelligence*, 2023.

[14] Bennedsen, J., et al. Novice programming students and the role of collaborative work. *ITiCSE Conference Proceedings*, 2019.

[15] Shute, V., Sun, C., & Clarke, J. Demystifying computational thinking. *Educational Research Review*, 22, 142–158, 2017.

[16] Guzdial, M. Computing education as a foundation for 21st century skills. *Communications of the ACM*, 2019.

[17] Hong Kong Education Bureau. IT in Education Policy: Promoting Computational Thinking. Available at: `https://www.edb.gov.hk`, 2019.

[18] Sentance, S., et al. Equity in computing education: Understanding teacher and student needs. *ACM Transactions on Computing Education*, 2019.

[19] Balik, M., et al. Comparative study of online IDEs for education. *Journal of Computing in Higher Education*, 2018.

[20] Wong, G., et al. School perceptions of coding education in K–12: Large scale study. *IEEE TALE Conference Proceedings*, 2017.

[21] Yadav, A., et al. Computational thinking in teacher education. *ACM Transactions on Computing Education*, 2016.

Chapter 6

Appendix A

Chapter 7

Appendix B

Chapter 8

Appendix C